

AI 사용 관련

아래와 같은 문제들에 대해 어떻게 대응, 대처해야 할까.

1. 검증(평가) 격차
2. 전문성(도메인 지식) 격차
3. Unknown unknowns
4. 신뢰 보정 문제
5. automation bias

핵심은 이거예요.

해결책은 “AI를 더 잘 믿는 법”이 아니라, AI가 틀려도 잡아낼 수 있는 외부 검증 구조를 만드는 것이에요.

사람들이 잘 대처하는 방식은 대체로 프롬프트 스킬보다 검증·통제·학습·관찰 시스템을 만드는 쪽에 가깝습니다.

1. 검증 격차 → “테스트와 평가 기준”을 먼저 만든다

초보자가 AI를 쓸 때 가장 많이 막히는 이유는, AI가 코드를 짜줘도 성공 조건이 뭔지 모른다는 거예요.

그래서 실무에서는 “일단 만들어줘”가 아니라 이렇게 접근합니다.

“이 작업이 제대로 됐다고 판단할 수 있는 기준은 무엇인가?”

“어떤 테스트를 통과해야 하는가?”

“어떤 로그, 그래프, 지표를 보면 실패를 알 수 있는가?”

AI agent 개발 쪽에서는 이걸 **evals, test harness, evaluation-driven development** 같은 식으로 다룹니다. LangChain 문서도 agent 테스트를 단위 테스트, 통합 테스트, trajectory evaluation으로 나누고, agent가 여러 컴포넌트를 연결하기 때문에 철저한 테스트가 필요하다고 설명합니다. Anthropic도 agent 평가는 코드 기반 채점, 모델 기반 채점, 사람 채점을 섞어서 쓰며, 회귀 테스트와 품질 평가를 따로 운영하는 패턴을 설명합니다. 강화학습으로 치면 “reward를 바꿔보자”보다 먼저 해야 하는 건 이런 것들이예요.

문제	검증 장치
환경이 제대로 구현됐나?	<code>reset</code> , <code>step</code> , termination 조건 unit test
reward가 의도대로 나오나?	상황별 reward table 테스트

문제	검증 장치
agent가 진짜 학습 중인가?	random agent, heuristic baseline, trained agent 비교
성능이 우연인가?	여러 seed 반복 실험
training만 잘하고 evaluation은 망하나?	별도 test environment에서 평가
reward hacking이 있나?	에피소드 영상, reward breakdown, 행동 로그 확인

Stable-Baselines3의 RL 팁도 “정량 실험”, “필요 시 하이퍼파라미터 튜닝”, “별도 테스트 환경에서 성능 평가”를 권장하고, RL은 agent가 환경과 상호작용하며 데이터를 직접 수집하기 때문에 나쁜 trajectory만 모이면 학습이 악순환에 빠질 수 있다고 설명합니다.

즉, **AI에게 답을 맡기는 게 아니라, AI가 낸 답이 통과해야 하는 시험지를 먼저 만드는 방식**이 예요.

2. 전문성 격차 → “전부 배우기”가 아니라 “판단할 만큼만 배우다”

사람들이 많이 쓰는 전략은 **해당 분야를 완전히 마스터한 뒤 AI를 쓰는 것**이 아니에요. 대신 **Minimum Viable Expertise**, 즉 “검증 가능한 수준의 최소 전문성”을 확보합니다.

예를 들어 강화학습을 할 때 당장 논문 수준까지 갈 필요는 없지만, 최소한 이 정도는 알아야 AI가 하는 말을 판단할 수 있어요.

알아야 하는 것	왜 필요한가
environment, state, action, reward, policy	프로젝트 구조를 이해하기 위해
episode return, success rate	학습이 되는지 판단하기 위해
exploration vs exploitation	agent가 왜 이상한 행동을 하는지 이해하기 위해
sparse reward, reward shaping	reward 설계 문제를 구분하기 위해
train/eval split, seed variance	결과를 과신하지 않기 위해
baseline, ablation	“개선됐다”는 말을 검증하기 위해

이건 “공부를 많이 하라”가 아니라, **AI의 산출물을 평가할 수 있는 해상도까지 도메인 지도를 만드는 것**에 가까워요.

Microsoft의 AI 과의존 리뷰에서도 사용자 특성, 전문성, task familiarity, AI literacy에 따라 AI를 과신하거나 과소신하는 방식이 달라지므로, 낮은 전문성의 사용자와 사용자의 자신감 수준을 특히 관찰해야 한다고 정리합니다.

실제로 잘하는 사람들은 AI에게 이렇게 묻습니다.

"이 분야에서 초보자가 반드시 알아야 할 핵심 개념 20%만 알려줘."

"각 개념을 모르면 프로젝트에서 어떤 증상으로 나타나는지 알려줘."

"내가 지금 하고 있는 작업에서 흔한 실패 모드 10개를 알려줘."

"각 실패 모드를 확인할 수 있는 실험이나 로그는 뭐야?"

AI를 개발자로만 쓰는 게 아니라, 먼저 **진단용 튜터**로 쓰는 거죠.

3. Unknown unknowns → "실패 모드 목록"을 만든다

"내가 뭘 모르는지도 모르는 상태"는 질문을 잘 못해서 생깁니다. 그래서 사람들은 **정답을 묻기 전에, 빠뜨렸을 가능성을 묻는 방식**을 씁니다.

좋은 패턴은 이거예요.

"내 계획에서 빠졌을 가능성이 큰 가정은 뭐야?"

"이 접근이 실패한다면 가장 그럴듯한 이유 5개는?"

"초보자가 이 문제에서 흔히 착각하는 것은?"

"지금 결과가 좋아 보여도 사실은 버그일 수 있는 경우는?"

"이 프로젝트를 리뷰하는 전문가라면 무엇부터 의심할까?"

이걸 **pre-mortem, failure-mode analysis, red-team review**처럼 부를 수 있어요.

강화학습 예시로는 이런 unknown unknowns가 많습니다.

초보자가 놓치기 쉬운 것	실제 문제
reward가 증가한다	그런데 agent가 의도와 다른 꼼수를 배웠을 수 있음
loss가 내려간다	RL에서는 supervised learning처럼 단순 해석하면 위험
한 seed에서 성공했다	다른 seed에서는 망할 수 있음
training env에서 잘한다	eval env나 초기 조건이 바뀌면 망할 수 있음
action이 움직인다	실제로는 random policy보다 나을 게 없을 수 있음
reward function이 그럴듯하다	reward leakage나 reward hacking이 있을 수 있음

그래서 잘하는 사람들은 AI에게 "답"보다 **의심 목록**을 먼저 뽑게 합니다.

4. 신뢰 보정 문제 → AI를 "항상 믿기/항상 의심하기"가 아니라 기록으로 보정한다

초보자는 보통 두 극단으로 갑니다.

하나는 **AI가 자신 있게 말하니까 믿는 것**이고, 다른 하나는 **뭐가 맞는지 모르니까 계속 불안해서 멈추는 것**이에요.

해결책은 감으로 믿는 게 아니라, **작은 calibration set**을 만드는 겁니다.

예를 들어 이런 식이에요.

작업 유형	AI 신뢰도 기록
문법적 코드 수정	대체로 신뢰 가능
라이브러리 사용법	공식 문서 확인 필요
RL reward 설계	반드시 실험으로 확인
성능 향상 원인 분석	낮은 신뢰, ablation 필요
논문식 설명	출처와 수식 확인 필요
실험 결과 해석	사람/전문가 리뷰 필요

사람들이 실무에서 하는 건 “이 모델은 똑똑하니까 믿자”가 아니라,

“이 종류의 작업에서 이 AI가 어느 정도 자주 맞았는가?”

“틀릴 때는 어떤 패턴으로 틀렸는가?”

“이 답은 테스트로 검증 가능한가, 아니면 전문가 판단이 필요한가?”

를 누적하는 방식이에요.

Microsoft의 Human-AI Interaction Guidelines도 AI 시스템이 무엇을 할 수 있는지, 얼마나 잘할 수 있는지, 틀렸을 때 어떻게 수정할 수 있는지, 불확실할 때 어떻게 범위를 좁히거나 gracefully degrade할지를 사용자에게 명확히 해야 한다고 제안합니다. Google의 People + AI Guidebook도 AI 제품에서는 사용자가 AI의 한계와 변화 방식을 이해하도록 staged onboarding, feedback, graceful failure, manual fallback을 설계하라고 설명합니다.

개인 사용자는 이걸 제품 설계가 아니라 자기 작업 방식에 적용하면 됩니다.

AI가 답을 주면 바로 믿지 말고,

“이 답을 검증할 수 있는 테스트는?”

“틀렸다면 어떤 관찰 결과가 나와야 하지?”

“내가 공식 문서나 실험으로 확인할 부분은?”

을 붙이는 거예요.

5. Automation bias → AI가 말하기 전에 내 판단을 먼저 적는다

automation bias는 AI 답을 먼저 보면 그 답을 기준으로 생각이 끌려가는 문제예요. Microsoft의 과의존 리뷰는 automation bias를 “자동화 시스템의 추천을 선호하고 비자동화 출처의 정보를 무시하는 경향”으로 설명하며, 특히 사용자가 충분한 지식과 기술이 없어 AI 추천을 평가하기 어려울 때 과의존이 생긴다고 정리합니다.

대응법은 의외로 단순합니다.

AI에게 묻기 전에 내 가설을 먼저 쓴다.

예를 들어 RL 프로젝트에서 학습이 안 될 때 바로 AI에게 “왜 안 돼?”라고 묻지 말고, 먼저 이렇게 적습니다.

내 가설 1: reward가 너무 sparse하다.

내 가설 2: termination 조건이 잘못됐다.

내 가설 3: action scaling이 틀렸다.

내 가설 4: training timesteps가 부족하다.

그다음 AI에게 묻습니다.

“내 가설들을 비판해줘. 더 가능성 높은 원인을 추가해줘. 각 가설을 검증할 최소 실험을 제안해줘.”

이렇게 하면 AI가 **생각의 출발점**이 아니라 **검토자**가 됩니다.

또 다른 방법은 **AI 역할 분리**예요.

첫 번째 AI에게는 구현을 맡기고, 두 번째 AI에게는 리뷰만 맡깁니다.

“아래 코드를 고치지 말고, 실패 가능성만 지적해.”

“이 reward function이 악용될 수 있는 방식을 찾아.”

“이 실험 설계에서 결론을 과장할 위험을 찾아.”

“초보자가 이 결과를 보고 잘못 해석할 부분을 알려줘.”

단, AI끼리 서로 검토하게 하는 것도 완전한 검증은 아니고, **테스트·문서·사람 리뷰를 보조하는 장치**로 봐야 합니다.

6. 사람들이 실제로 쓰는 대응 패턴

요즘 AI agent나 LLM 앱을 만드는 팀들은 대체로 이런 식으로 대응합니다.

첫째, **eval suite**를 만듭니다. agent가 잘해야 하는 대표 작업을 모아두고, 매번 프롬프트나 모델을 바꿀 때 같은 문제를 다시 풀게 합니다. Anthropic은 agent 평가에서 capability eval과 regression eval을 구분하고, 회귀 평가는 기존에 잘하던 일을 계속 잘하는지 확인하는 용도로 쓴다고 설명합니다.

둘째, **로그와 transcript를 읽습니다**. 점수만 보지 않고 agent가 어떤 tool을 불렀는지, 중간에 어떤 결정을 했는지, 어디서 잘못된 가정을 했는지를 확인합니다. Anthropic은 transcript를 읽어야 grader가 실제로 중요한 것을 측정하는지 확인할 수 있고, 실패가 agent 실수인지 평가 기준 문제인지 구분할 수 있다고 말합니다.

셋째, **human-in-the-loop**를 둡니다. 모든 것을 사람이 보는 게 아니라, 애매하거나 위험하거나 실패 비용이 큰 부분만 사람이 봅니다. NIST AI RMF도 AI 신뢰성 관련 지표와 threshold를 정할 때 인간 판단이 필요하고, subject matter expert가 평가 결과를 해석하고 배포 조건과 맞추는 데 도움을 줄 수 있다고 설명합니다.

넷째, **작게 자동화하고 점진적으로 권한을 늘립니다**. 처음부터 agent에게 전체 프로젝트를 맡기지 않고, "코드 제안만", "테스트 작성만", "로그 해석만", "한 파일 수정만"처럼 권한을 제한합니다. Google PAIR도 AI 제품에서는 자동화와 사용자 통제 사이의 균형이 중요하고, 사용자가 output을 수정하거나 끄거나 manual fallback을 쓸 수 있어야 한다고 설명합니다.

다섯째, **운영 후에도 모니터링합니다**. Anthropic은 agent 성능을 이해하려면 자동 평가뿐 아니라 production monitoring, user feedback, A/B testing, manual transcript review, systematic human evaluation이 함께 필요하다고 정리합니다.

7. 네 상황에 맞는 현실적인 접근법

강화학습을 AI agent로 개발한다면, 나는 이렇게 하길 추천해요.

1단계: AI에게 코드를 시키기 전에 "프로젝트 판정표"를 만들기

예를 들면:

이 RL 프로젝트의 성공 기준을 다음 형식으로 정리해줘.

1. 환경이 맞게 구현됐는지 확인하는 테스트
2. reward function sanity check
3. random policy baseline
4. heuristic baseline
5. training curve에서 봐야 할 지표
6. evaluation protocol
7. 실패했을 때 원인 후보와 진단 실험

이 단계에서는 코드를 만들지 말고 **검증 기준만** 만듭니다.

2단계: 가장 작은 toy 환경에서 시작하기

바로 복잡한 custom RL 환경으로 가지 말고, CartPole, LunarLander 같은 알려진 환경에서 agent가 학습되는 흐름을 먼저 봅니다.

목표는 "멋진 프로젝트 완성"이 아니라:

RL 실험이 정상일 때 어떤 로그와 그래프가 나오는지 몸에 익히기
예요.

3단계: custom environment는 unit test부터

AI에게 이런 식으로 시킵니다.

내 environment의 `reset()` 과 `step()` 이 Gymnasium/SB3 기준에 맞는지 테스트 코드를 작성해줘.

action 범위, observation shape, reward type, terminated/truncated 조건을 검증해 줘.

학습 코드는 아직 작성하지 마.

이게 중요합니다. RL은 알고리즘이 문제가 아니라 **환경 구현이 틀려서 학습이 안 되는 경우**가 정말 많아요.

4단계: random agent와 heuristic baseline을 먼저 돌리기

학습 agent보다 먼저 random agent를 돌려서 reward 분포를 봅니다.

random agent 평균 return은 얼마인가?

완전히 무작위인데도 reward가 높게 나오지는 않는가?

reward가 거의 항상 0이라 학습 신호가 없는 건 아닌가?

episode가 너무 빨리 끝나거나 끝나지 않는 건 아닌가?

그다음 가능하면 간단한 rule-based heuristic을 만듭니다. AI에게 학습 모델을 만들게 하기 전에, **사람이 이해할 수 있는 baseline**이 있어야 합니다.

5단계: AI 수정은 한 번에 하나만

초보자가 AI agent를 쓰면 망하기 쉬운 패턴이 이거예요.

reward 바꾸고, 네트워크 바꾸고, hyperparameter 바꾸고, observation 바꾸고, 알고리즘도 바꿈.

그러면 뭐 때문에 좋아졌는지/나빠졌는지 모릅니다.

대신 이렇게 해야 해요.

이번 실험에서는 reward scale만 바꾼다.

이번 실험에서는 learning rate만 바꾼다.

이번 실험에서는 observation 하나만 추가한다.

즉, AI에게도 이렇게 지시합니다.

“한 번에 하나의 변수만 바꿔. 바꾼 이유, 예상 효과, 실패 시 해석을 실험 로그에 남겨.”

이게 검증 격차를 줄이는 핵심입니다.

8. 다섯 문제별 해결책을 한 줄로 매핑하면

문제	대응
검증/평가 격차	테스트, eval, baseline, 로그, rubric을 먼저 만든다
전문성/도메인 지식 격차	전체를 공부하지 말고 “판단 가능한 최소 전문성”을 확보한다
Unknown unknowns	실패 모드, pre-mortem, 전문가 리뷰 질문을 만든다
신뢰 보정 문제	작업 유형별 AI 정확도 기록과 calibration set을 만든다
Automation bias	AI 답을 보기 전에 내 가설을 쓰고, AI는 검토자로 둔다

가장 실용적인 문장으로 압축하면 이거예요.

AI에게 “해결해줘”라고 하기 전에, “무엇이 맞는 해결인지 판정하는 방법부터 같이 만들자”고 해야 한다.

이게 AI 시대의 핵심 작업 방식에 가까워요. AI가 생성은 해줄 수 있지만, 방향을 잡고 검증 구조를 세우는 능력은 여전히 사람이 가져야 합니다.